CSE341 Assignment - 1

Name: Muaz Ibne Ashraf

ID: 21101122

Section: 04

(1) Segment registers of 8086

segment registers: CS, DS, SS, ES (code, data, stack, extra)

Purpose and operations

(i) 8086 uses segmented addressing.

Physical address = segment × 10h + offset.

(ii) Each segment register holds the starting address (upper 16 bits) of a 64 KB segment.

a) CS : IP ⇒ address of the next instruction.

b) DS + offset ⇒ default data accesses.

c) SS : SP/BP → stack access (push/pop, calls, interrupts)

d) ES → extra data segment.

So changing a segment register effectively relocates code/data/stack in memory without changing the program itself.

Difference from general purpose register:

general purpose is used for arithmatic, logic, loop counters, temporary data, addresses etc whereas

segment registers mainly used only in address generation, not for normal arithmatics. General purpose can be directly used by many ALU operations and segment registers can be modified only by specific instructions. and their contents are interpreted as segment bases, not ordinary data.

(ii) Given,

Segment Address = A4FB h

Offset = 4822h

Now,

Physical Address = (segment Address x10h) + offset

= (A4FB h + 10 h) + 4822h

= A4FB0h + 4822h

= A9822h . (Ans)

3)

1 MB memory space = 1024 KB

∴ segments of 64 KB that can be allocated

$$= \frac{1024}{64} = 16 \text{ (Ans)}$$

4) Given,

Physical Address = 32556 h

Largest offset = FFFFh

smallest offset = 0000h

5th highest:

(32556h − 0000h) = 32556h

Rounded down to nearest 10s = 3255h

∴ Largest segment = 3255h

5th largest = 3255h − 4h = 3251h

offset = 32556 − (32510h) → 3251 × 10h = 0046h

∴ Logical Address = 3251 : 0046 (Ans)

5th smallest:

(32556h - FFFFh) = 22552h

Rounded down to nearest 10's = 2256h

smallest segment = 2256h

5th smallest = 2256h + 4h = 225Ah

Offset = 32556h - (225Ah × 10h) = FF56h

∴ logical Address = 225A : FF56 (Ans)

5th

(i) Max size of memory supported = $2^{32}$ bytes

= 4294967296 bytes

= 4 GB. (Ans)

(ii) Yes, it is fully accessible. Because the memory can manage upto 4GB and the inserted memory is 3GB. Thus CPU can generate ~~upto~~ an address for every byte of the installed memory.

(iii) No, it is not fully accessible.

Because, the 4GB memory cannot manage more then 4GB so the remaining 2GB cannot addressed by CPU.

6 (i) Given PF = 0, AF = 1

Lower bytes : CDh + XYh

addition :

Max value of XY = FE (CDh + FEh = CBh → odd parity; PF = 0

D + E = carry; AF = 1)

PF = 0, AF = 0,

Max value of XY = F2 (CDh + F2h = BFh → odd parity, PF = 0

D + 2 = no unsigned overflow; AF = 0)

PF = 1, AF = 0;

Max value of XY = FF (CC → even parity, PF = 1

D + F = carry; AF = 1)

(ii) Minimum XY for same values :

PF = 0; AF = 1,

min XY = 03 (CD + 03) = D0 ⇒ odd parity; PF = parity

= 1 & AF = 1)

PF = 0; AF = 0D,

min XY = 00 (CD + 00 = CD → odd parity; PF = 0; output

⇒ AF = 0)

PF=1, AF=0;

min XY = 02 (CD+02=CF → even; PF=1, D+2=15<16, AF=0)

PF=1, AF=1,

min XY = 09 (CD+04=D1 → even; PF=1, D+4≥16, → AF=1)

21)

① ADD AX, DX → 9E40h + 5BC0h = BA00h

SF = 1 (as BA00h has, MSB=1, so, its negative)

CF = 0 (no carry out of bit 15)

PF = 1 (Low byte of the result is 00h, no "1" bit even parity)

OF = 1 (adding two positives → MSB 0 produce negatives → signed overflow)

AF = 0 (no carry for the low nibble bit)

ZF = 0 (result is not 0)

(10) ADD AX, BX → 00FFh + 0001h = 0100h

SF = 0 (No 1 in MSB, unsigned)

ZF = 0 (result not 0)

AF = 1 (even parity)

CF = 0 (no unsigned carry till 15th bit)

OF = 0 (no signed overflow)

11 a) Next instruction fetched using code segment (CS) and instruction pointer (IP)

Given CS = 2000h

IP = 3956h

∴ Physical address = (CS × 10h) + IP

= (2000h × 10h) + 3956h

= 23956h (Ans)

b) instruction = MOV DX, [BP + SI + 2000h]

offset = BP + SI + 2000h

= 2000h + 3000h + 2000h

= 2000h

$\therefore PA = (SS \times 10h) + offset$

$= (2000h \times 10h) + 2000h$

$= 22000h$

As 8086 Architecture is 'little-endian', lower address will be DX (DL)

$\therefore DL = 22000h = 56h$

$\therefore DH = 22001h = 20h$

$\therefore DX = DH : DL = 20:56 = 2056h.$

Ans

Q11

(i) Direct addressing

(ii) Register indirect addressing.

(iii) Indexed Addressing.

(iv) Based-Indexed addressing

(v) Direct addressing.

(vi) Register and implied addressing

(vii) Implied and register addressing.

(viii) Register indirect and indirect addressing

(IX) Implied and Register indirect addressing

(X) Direct addressing

(XI) Direct port and Register addressing

(XII) Direct port and Register addressing

(XIII) Indirect port and Register addressing.

(XIV) Indirect Port and " "

(XV) " " " " "

(XVI) Implied addressing ..

(XVII) " " .

(XVIII) " "

(XIX) " "

(XX) Implied addressing .

10) (1) MOV AX, [BX + 5634h]

opcode = 10001011 = 8Bh

Mod R/M Byte = 10001111 = 8Fh

Displacement = 5634h

Low byte = 34h

High byte = 56h

= 00110100 01010110

= 34h 56h

∴ Machine code = 10001011 10000111 00110100 01010110

Hex: 8B8734 56h

(4) OPcode = 10001001 = 89h = MOV mem, reg.

Mod R/M Byte = 10 000 111 = 87h = MOV [BX + d16], AX

Displacement = 9091h = MOV [BX + 9091h], AX

∴ Assembly instruction:

MOV [BX + 9091h], AX.